

第 10 章

关系查询处理和查询优化

第 10 章讲解了关系数据库管理系统 (RDBMS) 查询处理和查询优化的基本概念、方法和

技术。
查询优化是 RDBMS 内部实现的技术,对于一般用户而言是隐蔽的。用户不必了解 RDBMS 如何针对查询语句进行优化,然而对于 DBA 和应用开发人员来说,“性能调优 (performance tuning)”是其职责之一,因此需要掌握查询优化的概念和 RDBMS 的内部优化技术,进而为在数据库应用开发中利用查询优化技术提高查询效率和系统性能打下基础。

10.1 基本知识点

本章首先介绍 RDBMS 的查询处理步骤和实现查询操作的算法,然后深入讨论 RDBMS 查询优化的基本概念和方法,包括基于关系代数等价变换规则的优化方法、基于启发式规则的存取路径优化和基于代价估算的优化等方法。

查询处理是 RDBMS 的核心,它是 RDBMS 语言处理中最重要、最复杂的部分。

为了提高关系数据库系统的执行效率,RDBMS 必须进行查询优化。由于关系查询语言 (例如 SQL) 具有较高的语义层次,使得 RDBMS 可以进行查询优化,这就形成了 RDBMS 查询优化的必要性和可能性。

① 需要了解的:了解查询处理的基本步骤;了解查询分析、查询检查、查询优化和查询执行。

② 需要牢固掌握的:掌握什么是 RDBMS 的查询优化,查询优化的方法,特别是基于启发式规则的优化和基于代价估算的优化。

③ 需要举一反三的:能够画出一个查询的语法树以及优化后的语法树。

④ 难点:掌握查询优化算法,包括代数优化算法和物理优化算法。

10.2 习题解答和解析

1. 试述查询优化在关系数据库管理系统中的重要性和可能性。

【解析】请阅读《概论》第 10 章 10.2.1 小节“查询优化概述”相关内容。

查询优化的重要性: 查询优化既是 RDBMS 实现的关键技术, 又是 RDBMS 的优点所在。它减轻了用户选择存取路径的负担, 用户只要提出“干什么”, 而不必指出“怎么干”。

查询优化的优点不仅在于用户不必考虑如何最好地表达查询以获得较高的效率, 而且在于 RDBMS 可以比用户程序的“优化”做得更好。这是因为:

① 优化器可以从数据字典中获取许多统计信息, 例如每个关系表中的元组数, 关系中每个属性值的分布情况, 这些属性上是否建立了索引、是什么索引等。优化器可以根据这些信息做出正确的估算, 选择高效的执行计划, 而用户程序则难以获得这些信息。

② 如果数据库的物理统计信息改变了, 系统可以自动对查询进行重新优化以选择相适应的执行计划。在层次和网状数据库系统中, 这种情况必须重写程序, 而重写程序在实际应用中往往是不太可能的。

③ 优化器可以考虑数百种甚至数千种不同的执行计划, 并从中选出较优的一个, 而程序员一般只能考虑有限的几种可能性。

④ 优化器中包含很多复杂的优化技术, 特别是最新的机器学习技术。这些优化技术往往只有最好的程序员才能掌握。RDBMS 的查询自动优化相当于使得所有人都拥有了这些优化技术。

2. 假设关系 $R(A, B)$ 和 $S(B, C, D)$ 情况如下: R 有 20 000 个元组, S 有 1 200 个元组, 一个块能装 40 个 R 的元组、30 个 S 的元组, 试估算下列操作需要多少次磁盘块读写。

【解析】请阅读《概论》第 10 章 10.4.2 小节“基于代价估算的优化”相关内容。

① R 上没有索引, $\text{select } * \text{ from } R;$

② R 中 A 为主码, A 有 3 层 B+ 树索引, $\text{select } * \text{ from } R \text{ where } A = 10;$

③ 嵌套循环连接 $R \bowtie S$ 。

④ 排序合并连接 $R \bowtie S$, 区分 R 与 S 在 B 属性上有序和无序两种情况。

答:

① 需要对 R 进行全表扫描, 块数 $= 20\,000/40 = 500$ 。

② 对 R 进行索引扫描, 块数 $= 3 + 1 = 4$, 其中 3 块 B 树索引块, 1 块数据块。

③ R 本身有 $20\,000/40 = 500$ 个块, S 本身有 $1\,200/30 = 40$ 个块, 以 S 为外表, 假设内存分配的块数为 k , 嵌套循环连接需要的块数为 $40 + \left[\frac{40}{k-1} \right] \times 500$ 。

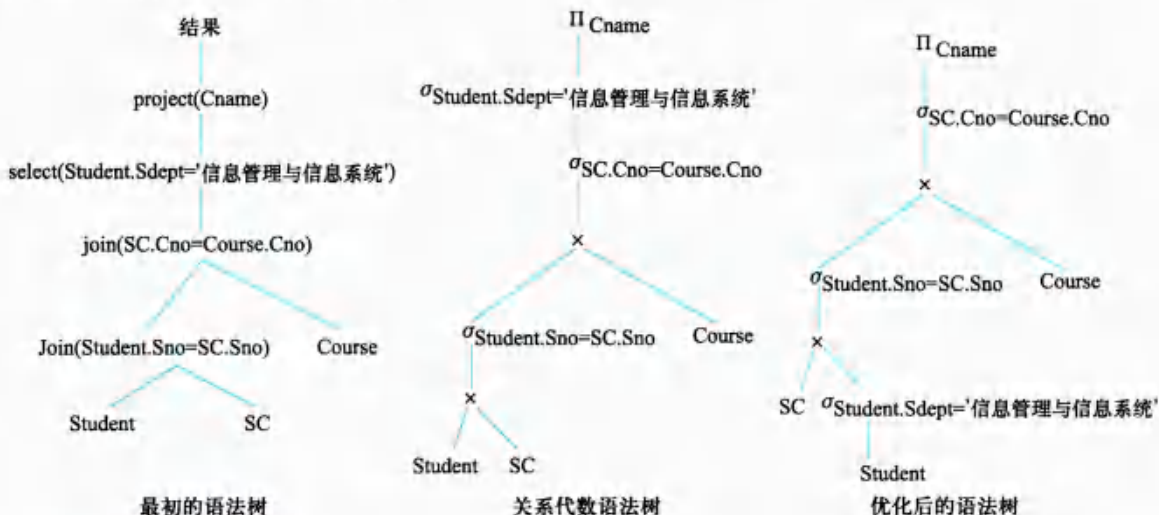
④ 如果 R 和 S 都在 B 属性上排好序, 块数为 $500 + 40 = 540$; 如果都没有排序, 则还要加上排序代价, 结果为 $540 + 2 \times 500 \times (\log_2 500 + 1) + 2 \times 40 \times (\log_2 40 + 1)$ 。

3. 对“学生选课管理”数据库, 查询信息管理与信息系统专业学生选修的所有课程名称。

```
SELECT Cname
FROM Student, Course, SC
```

WHERE Student.Sno=SC.Sno AND SC.Cno=Course.Cno AND Student.Smajor='IS';

试画出用关系代数表示的语法树，并用关系代数表达式优化算法对原始的语法树进行优化处理，画出优化后的标准语法树。



4. 对于数据库模式:

Teacher(Tno, Tname, Tage, Tsex)

Department(Dno, Dname, Tno)

Work(Tno, Dno, Year, Salary)

假设 Teacher 的 Tno 属性、Department 的 Dno 属性以及 Work 的 Year 属性上有 B+树索引，请说明下列查询语句的一种较优的处理方法。

- ① SELECT * FROM Teacher WHERE Tsex = '女';
- ② SELECT * FROM Department WHERE Dno < 301;
- ③ SELECT * FROM Work WHERE Year <> 2000;
- ④ SELECT * FROM Work WHERE Year > 2000 AND Salary < 5000;
- ⑤ SELECT * FROM Work WHERE Year < 2000 OR Salary < 5000;

【解析】

① 对 Teacher 进行全表扫描，查看元组是否满足性别为女。

因为男、女比例相差不会很大，使用全表扫描是合适的。

② 如果满足 Dno < 301 的元组数目较少，可以通过索引找到 Dno = 301 的索引项，然后顺着 B+树的顺序集得到 Dno < 301 的索引项，通过这些索引项的指针找到 Department 中的元组。

如果满足 Dno < 301 的元组数目较多，可以采用对 Department 的全表扫描方式处理。

RDBMS 的查询优化算法将计算这两种执行计划的代价，然后选择代价小的一种执行。

③ 对 Work 进行全表扫描，查看元组是否满足 $\text{Year} < 2000$ 。

虽然在 Work 的 Year 属性上有 B+树索引，因为要查找的是除 2000 年之外的所有元组，所以使用全表扫描是合适的。

④ 一种查询执行计划是：通过 Year 的索引找到满足 $\text{Year} > 2000$ 的元组，检查元组是否满足 $\text{Salary} < 5000$ 。

因为 Work 的 Year 属性上有 B+树索引，可以通过索引找到 $\text{Year}=2000$ 的索引项作为入口点，然后顺着 B+树的顺序集得到 $\text{Year} > 2000$ 的所有索引项，再通过这些索引项的指针找到 Work 元组中满足 $\text{Salary} < 5000$ 条件的元组。

另一种查询执行计划是：对 Work 进行全表扫描，一边扫描一边查看元组是否满足 $\text{Year} > 2000 \text{ AND } \text{Salary} < 5000$ 。

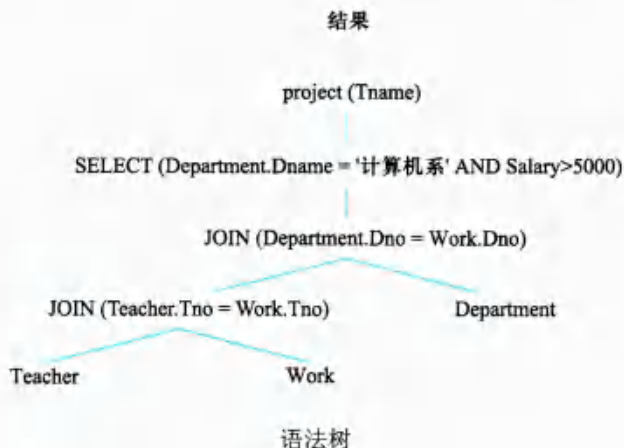
RDBMS 的查询优化算法将计算这两种执行计划的代价，然后选择代价小的一种执行。

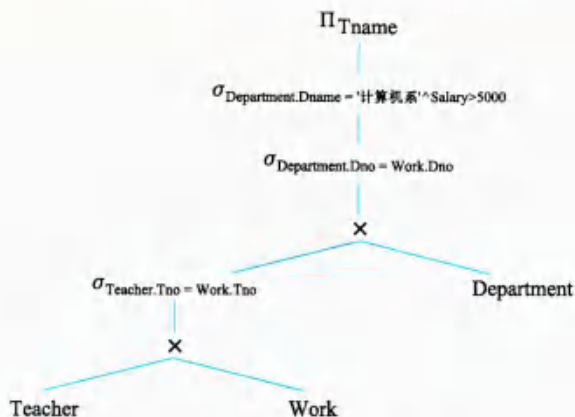
⑤ 对 Work 进行全表扫描，一边扫描一边查看元组是否满足 $\text{Year} < 2000$ 或 $\text{Salary} < 5000$ 。因为查询条件是 OR 连接的析取选择条件，所以采用全表扫描是合适的。

5. 对于第 4 题中的数据库模式，存在如下的查询：

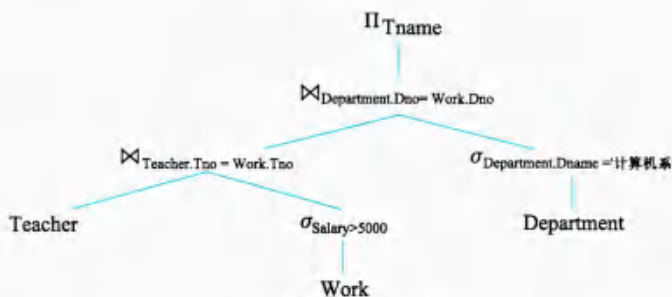
```
SELECT Tname
FROM Teacher, Department, Work
WHERE Teacher.Tno = Work.Tno AND Department.Dno = Work.Dno AND
      Department.Dname = '计算机系' AND Salary > 5000;
```

画出语法树以及用关系代数表示的语法树，并对关系代数语法树进行优化，画出优化后的语法树。





关系代数语法树



优化后的语法树

6. 试述关系数据库管理系统查询优化的一般准则。

【解析】请阅读《概论》第10章10.3.2小节“语法树的启发式优化”和10.4节“物理优化”相关内容。

下面的优化策略一般能提高查询效率：

- ① 选择运算应尽可能先做。
- ② 投影运算和选择运算同时进行。
- ③ 将投影同其前或其后的双目运算结合起来执行。
- ④ 将某些选择同在它前面要执行的笛卡儿积结合起来成为一个连接运算。
- ⑤ 找出公共子表达式。
- ⑥ 选取合适的连接算法。

其中，①~⑤是代数优化策略，⑥涉及物理优化。

① 选择运算应尽可能先做。因为满足选择条件的记录一般是原来关系的子集，从而使计算的中间结果变小。

② 投影运算和选择运算同时进行。如果在同一个关系上有若干投影和选择运算，则可以把投影运算和选择运算结合起来，即选出符合条件的记录后就对这些元组做投影。

③ 把投影同其前或其后的双目运算结合起来。双目运算有 JOIN 运算、笛卡儿积运算。与上面的理由类似,在进行 JOIN 运算、笛卡儿积运算时要选出关系的元组,没有必要为了投影操作(通常是去掉某些字段)而单独扫描一遍关系。

④ 把某些选择同在它前面要执行的笛卡儿积运算结合起来成为一个连接运算。连接运算,特别是等连接运算要比在同样关系上的笛卡儿积运算产生的结果小得多,执行代价也小得多。

⑤ 找出公共子表达式。先计算一次公共子表达式并把结果保存起来共享,以避免重复计算公共子表达式。当查询的是视图时,定义视图的表达式就是公共子表达式的情况。

⑥ 选取合适的连接算法。连接操作是关系操作中最费时的操作,人们研究了许多连接优化算法。例如,索引连接算法、排序—合并算法、哈希连接算法等。

选取合适的连接算法属于选择“存取路径”,是物理优化的范畴。

7. 试述关系数据库管理系统查询优化的一般步骤。

【解析】各个关系数据库管理系统的优化方法不尽相同,大致的步骤可以归纳如下:

① 把查询转换成某种内部表示,通常用的内部表示是语法树。

② 把语法树转换成标准(优化)形式,即利用优化算法,把原始的语法树转换成优化的形式。

③ 选择低层的存取路径。

④ 生成查询计划,选择所需代价最小的计划加以执行。

读者要把 SQL 查询处理和查询优化的概念结合起来学习,了解 SQL 查询处理工作的步骤,如《概论》第 10 章图 10.1 所示,了解查询优化在查询处理中的核心地位。

10.3 补充习题及答案

考虑关系模式 $R(A, B, C, D)$, 假设 B 上具有 B+树索引, (C, D) 上具有聚簇的 B+树索引。每条关系记录占 100 字节,索引的数据条目占 20 字节,整个关系表占 10 000 个数据块。假设符合每个条件的数目比例为 10%, B+树为 3 层,每个数据块包括 40 个关系记录,计算执行下列语句的开销,选出较小的执行代价。

① `SELECT * FROM R WHERE B > 1000;`

② `SELECT * FROM R WHERE C = 10;`

③ `SELECT * FROM R WHERE C = 20 AND D > 100;`

④ `SELECT SUM(B) FROM R WHERE B > 1000;`

【解析】

① `SELECT * FROM R WHERE B > 1000;`

第一种查询计划是使用 B+树索引,代价为

$(B+树前两层) + (B+树叶结点的块数) + 存取表中记录需要读取的块数$